

Modifiers and encapsulation

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA



Miller Trujillo
Sr Software Engineer

Access modifiers

Can access from...

Modifier	Within class	A class from same package	A class from a different package
Public	Yes	Yes	Yes
Protected	Yes	Yes	Yes, if it is a subclass
Default	Yes	Yes	No
Private	Yes	No	No

Access modifiers: Example

```
class Dog {  
    private String name;  
    private int age;  
    private String bark;  
    public Dog(String name, int age) { // Constructor  
        this.name = name;  
        this.age = age;  
        this.bark = "Woof!";  
    }  
    public void bark() { // Public method  
        System.out.println(bark + " name = " + name + ", age = " + age);  
    }  
}
```

Private method

```
class Dog {  
    ...  
    public void bark() { // Public method  
        System.out.println(prepareBark());  
    }  
  
    private String prepareBark() { // Helper method that remains private  
        return bark + " name = " + name + ", age = " + age;  
    }  
}
```

Encapsulation

- Encapsulation is a key principle of OOP

```
class Counter {  
    private int count; // Field  
    public void increase() { // Allowed behavior  
        this.count++;  
    }  
    public int getCount() { // Getter  
        return this.count;  
    }  
    public void setCount(int count) { // Setter  
        this.count = count;  
    }  
}
```

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Extending a class with inheritance

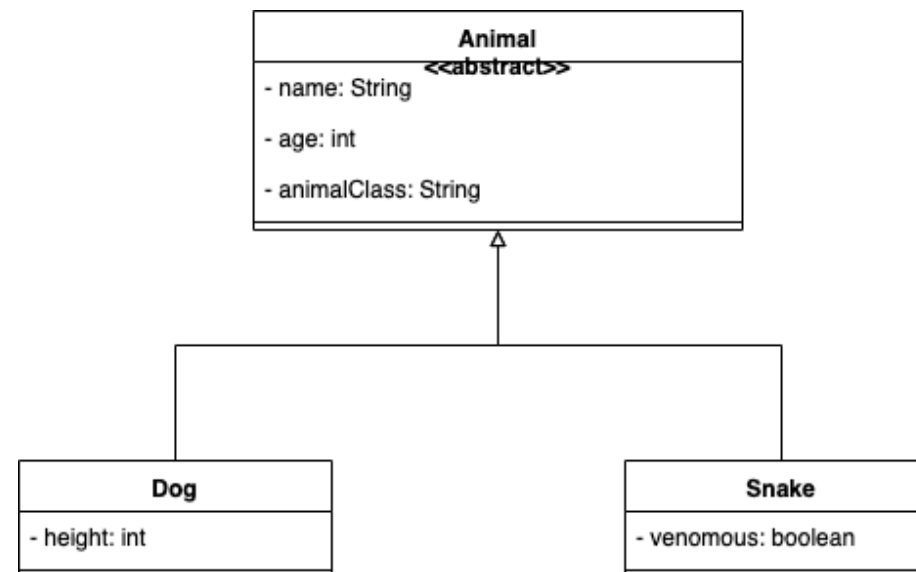
INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA



Miller Trujillo
Sr Software Engineer

What is inheritance?

- Base a class from another
- Hierarchies
- Share common attributes (fields) or behaviors (methods)
- Reuse code
- Flexibility



Inheritance concepts

- Parent class == Superclass
- Child class == Subclass
- Inheritance allows sharing
- Subclass can utilize all the fields and methods from the parent class*

Protected

- `protected` = accessible only to child classes and the class itself

```
public class Animal {  
    protected String name;  
    protected int age;  
}
```

- `private` only allow access within the class, not subclasses.

extends

```
public class Animal {  
    protected name;  
    protected age;  
}  
  
public class Dog extends Animal {  
    private String breed;  
  
    public void printDetails() {  
        System.out.println(" name = " + name + ", age = " + age + ", breed = " + breed);  
    }  
}
```

super

```
public class Animal {  
    protected name;  
    public Animal(String name) { // Superclass constructor  
        this.name = name;  
    }  
}  
  
public class Dog extends Animal {  
    private String breed;  
    public Dog(String name, String breed) { // Subclass constructor  
        super(name); // Superclass constructor call  
        this.breed = breed;  
        System.out.println(" name = " + super.name + ", breed = " + breed);  
    }  
}
```

Recap

- `extends` is used to define an inheritance, from the subclass referencing superclass.
 - `public class B extends A`
- `protected` is used to restrict access to fields or methods to only subclasses
- `super` is used to reference superclass (parent class)
 - `super()` -> calls parent constructor

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Inheritance: Abstract & abstraction

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

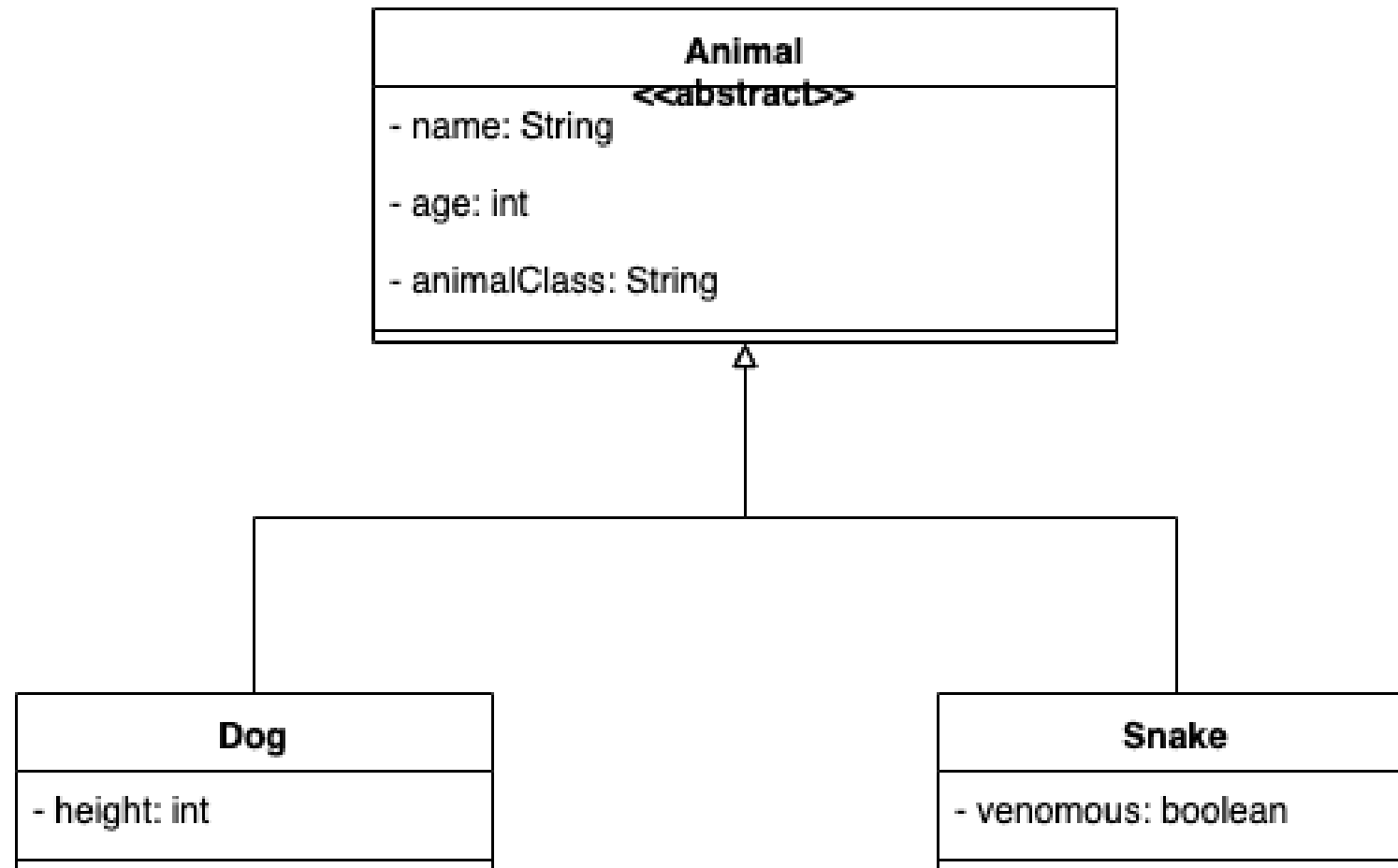


Miller Trujillo
Sr Software Engineer

What is an abstract class?

- Abstract classes **don't** allow object creation
- Subclasses can extend abstract classes
- Model higher-level concepts
- Better organize and structure our code

Abstract class example



```
public abstract class Animal { // Abstract
    protected String name;
    protected int age;
    protected String animalClass;
}

public class Dog extends Animal { // Extended
    private int height;
}

public class Snake extends Animal { // Extended
    private boolean venomous;
}
```

Why use abstract classes?

- Restrict the possibility to create Animal object if not needed
- Abstraction: Hiding the unnecessary details from those who don't need to know about it.
- Prevent confusions: Clear expectations
- Allow mutability of the implementation details

Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA

Interfaces: implements

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA



Miller Trujillo
Sr Software Engineer

What is an interface?

- Method signature: `void myMethod(String myParameter, String mySecondParameter)`
- The signature uniquely identifies a method within a class
- Set of method signatures: A contract
- Classes that adhere to the contract must implement those methods

How to create an interface in Java?

```
public interface Animal { // Interface definition
    String getSound(); // Method signature
}
```

- Methods are public by default

Implementing an interface

```
public class Dog implements Animal { // Implementing the interface
    @Override // Override the interface method
    public int getSound() {
        return "Woof!";
    }
}
```


Subtyping

```
public class Main {  
    public static void main(String[] args) {  
        Animal a1 = new Dog();  
        System.out.println(a1.getSound()); // Woof!  
        a1.getName(); // Will fail  
    }  
}
```


Let's practice!

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN JAVA